06. 컨센서스 모드 전환 메커니즘

목차

1. 모드 전환 개념
2. 전환 조건 평가
3. 안전한 모드 전환
4. 네트워크 조건 모니터링
5. 전환 기록 및 추적
6. 복구 메커니즘

모드 전환 개념

Autobahn BFT의 핵심 기능 중 하나는 네트워크 조건과 시스템 상태에 따라 Fast Path와 Slow Path 간을 동적으로 전환하는 능력입니다. 이 메커니즘은 성능과 안전성 사이의 균형을 유지합니다.

컨센서스 모드 정의

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]

pub  enum ConsensusMode {

FastPath, // 빠른 경로: 정상 조건에서 최적 성능

SlowPath, // 느린 경로: 비잔틴 장애 허용 보장

Transitioning, // 전환 중: 모드 변경 과정

}
```

```
impl ConsensusMode {
pub fn allows_fast_path(&self) -> bool {
matches!(self, ConsensusMode::FastPath)
}
pub fn valid_transitions(&self) -> Vec<ConsensusMode> {
match self {
ConsensusMode::FastPath => vec![ConsensusMode::SlowPath, ConsensusMode::Transiti
ConsensusMode::SlowPath => vec![ConsensusMode::FastPath, ConsensusMode::Transiti
ConsensusMode::Transitioning => vec![ConsensusMode::FastPath, ConsensusMode::Slo
}
}
pub fn can_transition_to(&self, target: ConsensusMode) -> bool {
self.valid_transitions().contains(&target)
}
}
```

전환 이유 정의

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum TransitionReason {
ByzantineDetected, // 비잔틴 행동 감지
FastPathTimeout, // Fast Path 타임아웃
```

```
NetworkDegradation, // 네트워크 성능 저하
AutomaticRecovery, // 자동 복구
ManualOverride, // 수동 재정의
PerformanceOptimization, // 성능 최적화
}
```

전환 조건 평가

AutobahnController의 모드 전환 평가

```
impl AutobahnController {
pub  async  fn  evaluate_mode_transition(&self) -> Result<Option<ConsensusMode>,
let  current_mode = self.get_current_mode().await;
let  metrics = self.get_metrics().await;
let  timeout_stats = self.get_timeout_stats().await;

match  current_mode {
ConsensusMode::FastPath => {
// Slow Path로 폴백해야 하는지 확인
if  self.should_fallback_to_slow_path(&metrics, &timeout_stats).await? {
Ok(Some(ConsensusMode::SlowPath))
} else {
Ok(None)
```

```
}
}
ConsensusMode::SlowPath => {
// Fast Path로 복구할 수 있는지 확인
if  self.can_recover_to_fast_path(&metrics, &timeout_stats).await? {
Ok(Some(ConsensusMode::FastPath))
} else {
Ok(None)
}
}
ConsensusMode::Transitioning => {
// 조건에 따라 목표 모드 결정
if  self.should_fallback_to_slow_path(&metrics, &timeout_stats).await? {
Ok(Some(ConsensusMode::SlowPath))
} else {
Ok(Some(ConsensusMode::FastPath))
}
}
}
}
}
```

Fast Path에서 Slow Path로 폴백 조건

```
impl AutobahnController {
async fn should_fallback_to_slow_path(
&self,
metrics: &AutobahnMetrics,
timeout_stats: &TimeoutStats,
) -> Result<bool, AutobahnError> {
// 1. 비잔틴 행동 감지 확인
if self.is_byzantine_detected().await {
return Ok(true);
}

// 2. 과도한 타임아웃 확인
if timeout_stats.consecutive_timeouts >= 3 {
return Ok(true);
}

// 3. Fast Path 성공률 확인
let performance_metrics = metrics.performance_metrics();
if performance_metrics.fast_path_success_rate < 0.5
&& (performance_metrics.fast_path_usage_count + performance_metrics.slow_path_us
{
```

```
return Ok(true);
}

// 4. 네트워크 조건 확인
if let Some(avg_latency) = timeout_stats.avg_latency {
if avg_latency > timeout_stats.current_timeout / 2 {
return Ok(true);
}
}

Ok(false)
}
}
```

Slow Path에서 Fast Path로 복구 조건

```
impl AutobahnController {
async fn can_recover_to_fast_path(
&self,
metrics: &AutobahnMetrics,
timeout_stats: &TimeoutStats,
) -> Result<bool, AutobahnError> {
// 1. 비잔틴 행동이 감지된 경우 복구하지 않음
```

```
if self.is_byzantine_detected().await {
return Ok(false);
}

// 2. 네트워크 조건이 개선되었는지 확인
if timeout_stats.consecutive_timeouts == 0 {
if let Some(avg_latency) = timeout_stats.avg_latency {
// 네트워크 지연시간이 합리적이어야 함
if avg_latency < timeout_stats.base_timeout / 4 {
return Ok(true);
}
}
}

// 3. Slow Path에서 충분한 시간 동안 안정화되었는지 확인
let mode_transitions = metrics.mode_transition_history();
if let Some(last_transition) = mode_transitions.last() {
let (timestamp, from_mode, to_mode) = last_transition;
if *to_mode == ConsensusMode::SlowPath {
// 복구를 고려하기 전에 최소 10초 대기
let now = crate::types::Timestamp::now();
if now >= *timestamp {
return Ok(true);
```

```
}
}
}

Ok(false)
}
}
```

안전한 모드 전환

모드 전환 실행

```
impl AutobahnController {
pub  async  fn  execute_mode_transition(
&mut  self,
target_mode: ConsensusMode,
) -> Result<(), AutobahnError> {
let  current_mode = self.get_current_mode().await;

if  current_mode == target_mode {
return Ok(()); // 이미 목표 모드
}
```

```
// 전환이 허용되는지 검증
if !current_mode.can_transition_to(target_mode) {
return Err(AutobahnError::ModeTransitionFailed {
from: current_mode,
to: target_mode,
reason: "Invalid mode transition".to_string(),
});
}


// 먼저 전환 상태로 설정
{
let mut mode = self.current_mode.write().await;
*mode = ConsensusMode::Transitioning;
}


// 목표 모드에 따른 전환 로직 실행
match target_mode {
ConsensusMode::FastPath => {
self.transition_to_fast_path().await?;
}
ConsensusMode::SlowPath => {
self.transition_to_slow_path().await?;
}
```

```
ConsensusMode::Transitioning => {
// 이미 전환 상태
return Ok(());
}
}

// 최종 모드 업데이트
{
let mut mode = self.current_mode.write().await;
*mode = target_mode;
}

// 메트릭과 상태에 전환 기록
self.record_mode_transition(current_mode, target_mode).await?;

Ok(())
}
}
```

Fast Path로 전환

```
impl AutobahnController {
async fn transition_to_fast_path(&mut self) -> Result<(), AutobahnError> {
```

```
// Slow Path에서 비잔틴 감지 리셋
{
let mut slow_path = self.slow_path.write().await;
slow_path.set_byzantine_detected(false).await;
}

// 새로운 시작을 위해 타임아웃 매니저 리셋
{
let mut timeout_manager = self.timeout_manager.write().await;
timeout_manager.reset_timeout();
}

// 오래된 집계 메시지 정리
{
let mut aggregator = self.message_aggregator.write().await;
aggregator.cleanup_old_messages(5, types::Round::default());
}

Ok(())
}
}
```

Slow Path로 전환

```rust
impl AutobahnController {
async fn transition_to_slow_path(&mut self) -> Result<(), AutobahnError> {
// 전환 이유 결정
let reason = if self.is_byzantine_detected().await {
TransitionReason::ByzantineDetected
} else {
let timeout_stats = self.get_timeout_stats().await;
if timeout_stats.consecutive_timeouts >= 3 {
TransitionReason::FastPathTimeout
} else {
TransitionReason::NetworkDegradation
}
};

// Slow Path에서 폴백 트리거
{
let mut slow_path = self.slow_path.write().await;
slow_path.trigger_automatic_fallback(reason).await?;
}

// Slow Path 작업을 위한 타임아웃 증가
{
```

```
let mut timeout_manager = self.timeout_manager.write().await;
timeout_manager.increase_timeout();
}

Ok(())
}
}
```

전환 기록

```
impl AutobahnController {
async fn record_mode_transition(
&self,
from: ConsensusMode,
to: ConsensusMode,
) -> Result<(), AutobahnError> {
// 메트릭 업데이트
{
let mut metrics = self.metrics.write().await;
metrics.record_mode_transition(from, to);
}

// 컨센서스 상태 업데이트
```

```
{
let mut state = self.autobahn_state.write().await;
let reason = match to {
ConsensusMode::SlowPath => TransitionReason::ByzantineDetected,
ConsensusMode::FastPath => TransitionReason::AutomaticRecovery,
ConsensusMode::Transitioning => TransitionReason::ManualOverride,
};
state.record_transition(from, to, reason);
}

Ok(())
}
}
```

네트워크 조건 모니터링

네트워크 조건 구조

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum NetworkCondition {
Good, // 양호한 조건
Degraded, // 성능 저하
Poor, // 열악한 조건
```

```
Unknown, // 알 수 없음
}

#[derive(Debug, Clone)]
pub struct NetworkConditions {
pub latency: NetworkCondition,
pub timeouts: NetworkCondition,
pub throughput: NetworkCondition,
pub overall: NetworkCondition,
}
```

네트워크 조건 모니터링

```
impl AutobahnController {
pub async fn monitor_network_conditions(&self) -> Result<NetworkConditions, A
let timeout_stats = self.get_timeout_stats().await;
let aggregation_stats = self.get_aggregation_stats().await;

let latency_condition = if let Some(avg_latency) = timeout_stats.avg_latency {
if avg_latency > timeout_stats.base_timeout {
NetworkCondition::Poor
} else if avg_latency > timeout_stats.base_timeout / 2 {
NetworkCondition::Degraded
```

```
} else {
NetworkCondition::Good
}
} else {
NetworkCondition::Unknown
};

let timeout_condition = if timeout_stats.consecutive_timeouts >= 3 {
NetworkCondition::Poor
} else if timeout_stats.consecutive_timeouts >= 1 {
NetworkCondition::Degraded
} else {
NetworkCondition::Good
};

let throughput_condition = if aggregation_stats.average_batch_size < 2.0 {
NetworkCondition::Poor
} else if aggregation_stats.average_batch_size < 5.0 {
NetworkCondition::Degraded
} else {
NetworkCondition::Good
};
```

```
Ok(NetworkConditions {
latency: latency_condition,
timeouts: timeout_condition,
throughput: throughput_condition,
overall: Self::calculate_overall_condition(
latency_condition,
timeout_condition,
throughput_condition,
),
})
}
}
```

전체 네트워크 조건 계산

```
impl AutobahnController {
fn  calculate_overall_condition(
latency: NetworkCondition,
timeouts: NetworkCondition,
throughput: NetworkCondition,
) -> NetworkCondition {
let  conditions = [latency, timeouts, throughput];
```

```
// 어떤 조건이라도 열악하면 전체가 열악함
if  conditions.iter().any(|c| matches!(c, NetworkCondition::Poor)) {
return NetworkCondition::Poor;
}

// 과반수가 성능 저하면 전체가 성능 저하
let  degraded_count = conditions
.iter()
.filter(|c| matches!(c, NetworkCondition::Degraded))
.count();

if  degraded_count >= 2 {
return NetworkCondition::Degraded;
}

// 모두 양호하거나 대부분 양호하면 전체가 양호
let  good_count = conditions
.iter()
.filter(|c| matches!(c, NetworkCondition::Good))
.count();

if  good_count >= 2 {
NetworkCondition::Good
```

```
} else {
NetworkCondition::Unknown
}
}
}
```

전환 기록 및 추적

비잔틴 행동 감지

```
impl AutobahnController {
pub  async  fn  detect_byzantine_behavior(&self) -> Result<bool, AutobahnError>
// Slow Path 비잔틴 감지 확인
let  slow_path_byzantine = {
let  slow_path = self.slow_path.read().await;
slow_path.is_byzantine_detected().await
};

if  slow_path_byzantine {
return Ok(true);
}

// Fast Path 실행 가능성 확인
```

```
let fast_path_viability = {
let fast_path = self.fast_path.read().await;
fast_path.evaluate_fast_path_viability()
};


match fast_path_viability {
crate::fast_path::FastPathViability::ShouldFallback { confidence, .. } => {
if confidence > 0.8 {
return Ok(true);
}
}
_ => {}
}


// 비잔틴 패턴에 대한 메트릭 확인
let metrics = self.get_metrics().await;
let mode_transitions = metrics.mode_transition_history();


// 너무 많은 빠른 전환은 비잔틴 행동을 나타낼 수 있음
if mode_transitions.len() >= 5 {
let recent_transitions: Vec<_> = mode_transitions.iter().rev().take(5).collect(
let byzantine_transitions = recent_transitions
.iter()
```

```
.filter(|(_, from, to)| {
matches!(
(from, to),
(ConsensusMode::FastPath, ConsensusMode::SlowPath)
| (ConsensusMode::SlowPath, ConsensusMode::FastPath)
)
})
.count();

if byzantine_transitions >= 3 {
return Ok(true);
}
}

Ok(false)
}
}
```

모드 전환 히스토리 추적

```
impl AutobahnMetrics {
pub fn record_mode_transition(&mut self, from: ConsensusMode, to: ConsensusMo
let timestamp = crate::types::Timestamp::now();
```

```
self.mode_transitions.push((timestamp, from, to));

// 히스토리 크기 제한 (최근 100개 전환만 유지)

if  self.mode_transitions.len() > 100 {

self.mode_transitions.remove(0);

}

// 전환 통계 업데이트

match (from, to) {

(ConsensusMode::FastPath, ConsensusMode::SlowPath) => {

self.fast_to_slow_transitions += 1;

}

(ConsensusMode::SlowPath, ConsensusMode::FastPath) => {

self.slow_to_fast_transitions += 1;

}

_ => {}

}

}

pub  fn  mode_transition_history(&self) -> &[(crate::types::Timestamp, Consensus

&self.mode_transitions

}

}
```

복구 메커니즘

자동 복구 로직

```
impl AutobahnController {
pub  async  fn  attempt_automatic_recovery(&mut  self) -> Result<bool, AutobahnE
let  current_mode = self.get_current_mode().await;
// Fast Path에서만 복구 시도
if  current_mode != ConsensusMode::SlowPath {
return Ok(false);
}
let  network_conditions = self.monitor_network_conditions().await?;
let  metrics = self.get_metrics().await;
// 복구 조건 확인
let  can_recover =
network_conditions.overall == NetworkCondition::Good &&
!self.is_byzantine_detected().await &&
self.has_stable_performance(&metrics).await?;
if  can_recover {
tracing::info!("Attempting automatic recovery to Fast Path");
self.execute_mode_transition(ConsensusMode::FastPath).await?;
Ok(true)
} else {
Ok(false)
}
```

```
}

async fn has_stable_performance(&self, metrics: &AutobahnMetrics) -> Result<bo

let transitions = metrics.mode_transition_history();

// 최근 5분 동안 전환이 없었는지 확인

let recent_threshold = crate::types::Timestamp::now() - Duration::from_secs(300

let recent_transitions = transitions

.iter()

.filter(|(timestamp, _, _)| *timestamp > recent_threshold)

.count();

Ok(recent_transitions == 0)

}

}
```

강제 모드 전환 (관리자 개입)

```
impl AutobahnController {

pub async fn force_mode_transition(

&mut self,

target_mode: ConsensusMode,

reason: String,

) -> Result<(), AutobahnError> {

let current_mode = self.get_current_mode().await;

tracing::warn!(
```

```
"Forcing mode transition from {:?} to {:?}: {}",
current_mode,
target_mode,
reason
);
// 강제 전환은 유효성 검사를 우회
{
let mut mode = self.current_mode.write().await;
*mode = target_mode;
}
// 전환 기록
self.record_mode_transition(current_mode, target_mode).await?;
// 메트릭에 강제 전환 표시
{
let mut metrics = self.metrics.write().await;
metrics.record_forced_transition(current_mode, target_mode, reason);
}
Ok(())
}
}
```

전환 실패 처리

```
impl AutobahnController {
async fn handle_transition_failure(
&mut self,
from: ConsensusMode,
to: ConsensusMode,
error: AutobahnError,
) -> Result<(), AutobahnError> {
tracing::error!(
"Mode transition failed from {:?} to {:?}: {:?}",
from,
to,
error
);
// 안전한 상태로 복구 시도
match from {
ConsensusMode::FastPath => {
// Fast Path에서 실패한 경우 Slow Path로 강제 전환
self.force_mode_transition(
ConsensusMode::SlowPath,
"Recovery from failed transition".to_string(),
).await?;
}
```

```
ConsensusMode::SlowPath => {
// Slow Path는 안전한 상태이므로 유지
tracing::info!("Maintaining Slow Path mode after transition failure");
}
ConsensusMode::Transitioning => {
// 전환 중 실패한 경우 Slow Path로 안전하게 복구
self.force_mode_transition(
ConsensusMode::SlowPath,
"Recovery from transitioning state failure".to_string(),
).await?;
}
}
Ok(())
}
}
```

모드 전환 최적화 전략

전환 빈도 제한

```
impl AutobahnController {
async fn should_limit_transitions(&self) -> bool {
let metrics = self.get_metrics().await;
```

```
let transitions = metrics.mode_transition_history();
// 최근 1분 동안 3회 이상 전환이 있었다면 제한
let recent_threshold = crate::types::Timestamp::now() - Duration::from_secs(60)
let recent_count = transitions
.iter()
.filter(|(timestamp, _, _)| *timestamp > recent_threshold)
.count();
recent_count >= 3
}
}
```

예측적 전환

```
impl AutobahnController {
async fn predictive_mode_transition(&self) -> Result<Option<ConsensusMode>, Au
let network_conditions = self.monitor_network_conditions().await?;
let current_mode = self.get_current_mode().await;
// 네트워크 조건이 악화되고 있는 추세인지 확인
let degradation_trend = self.detect_degradation_trend().await?;
match (current_mode, network_conditions.overall, degradation_trend) {
(ConsensusMode::FastPath, NetworkCondition::Degraded, true) => {
// 성능 저하 추세가 감지되면 사전에 Slow Path로 전환
Ok(Some(ConsensusMode::SlowPath))
```

```
}

(ConsensusMode::SlowPath, NetworkCondition::Good, false) => {

// 네트워크가 안정되면 Fast Path로 복구

Ok(Some(ConsensusMode::FastPath))

}

_ => Ok(None),

}

}

async fn detect_degradation_trend(&self) -> Result<bool, AutobahnError> {

let timeout_stats = self.get_timeout_stats().await;

// 연속 타임아웃이 증가하는 추세인지 확인

Ok(timeout_stats.consecutive_timeouts > 0)

}

}
```

컨센서스 모드 전환 메커니즘은 Autobahn BFT의 적응성과 복원력을 제공하는 핵심 기능으로, 네트워크 조건 변화에 동적으로 대응하여 최적의 성능과 안전성을 보장합니다.